GDPR Article 17 — Right to Erasure

When a customer asks to be deleted, most pipelines scramble: someone runs manual `DELETE`s across a handful of tables, hopes nothing downstream rebuilt the data back, and keeps no record they did it. This pipeline treats erasure as a **standing, auditable, idempotent control** — one row in a registry, and the pipeline enforces it on every run.

It implements two distinct controls, deliberately kept separate:

Control	What it does	Where it acts
Suppression	Erased customers are gated out of all downstream activation (e.g. the Klaviyo audience). They can no longer be marketed to.	`stg_activation_pii` → reverse-ETL view
Destruction (true erasure)	The identifying PII itself is irreversibly replaced at the source layer. There is no original left to recover.	`RAW.CUSTOMERS`

Most pipelines do — at best — suppression, and call it "deletion." Article 17 asks for destruction: the data subject can no longer be identified *at all*. This pipeline does both.

How it works

1. The erasure registry (the audit trail). A single governed table, `gdpr_erasure_registry`, records every request: `customer_id`, `request_date`, `erased_at`, `erased_by`, and `status` (`requested` → `erased`). This is the source of truth — and the record you hand a DPO.

2. Suppression — enforced declaratively. `stg_activation_pii` derives `IS_ERASED` directly from the registry. The reverse-ETL audience already gates on `IS_ERASED = FALSE`, so an erased customer falls out of the Klaviyo sync automatically on the next run — no `UPDATE`, no manual step, idempotent by construction. A customer marked `requested` (not yet `erased`) is *not* suppressed, proving the registry is a real state machine, not a binary flag.

3. Destruction — irreversible at source. `erase_at_source` replaces each PII column in `RAW.CUSTOMERS` with `SHA2(customer_id || ':' || column, 256)` for registry members. The hash is keyed to the internal surrogate ID — *not* the email — so it cannot be reverse-looked-up from a guessed email, and there is no plaintext left. An idempotency guard skips already-erased rows, so the operation is safe to run on every load.

4. Reconciliation — the pipeline proves its own compliance. `gdpr_erasure_audit` joins the registry against the live activation layer and flags any mismatch: an `erased` record that *isn't* suppressed would surface as `MISMATCH — investigate`. The log doesn't just claim erasure happened; it continuously verifies the live state matches the registry.

Proof (demo data)

A hero customer (`CUST00028`) — consented, active, and in the live 1,065-row Klaviyo audience — is added to the registry as `erased`:

- **Suppression:** audience drops 1,065 → 1,064; the hero is gone, while two `requested`-only customers remain (request ≠ action).

- **Destruction:** `RAW.CUSTOMERS.email` changes from `oliver.cust00028[at]example[dot]com` to `751b9615…2957bd6`; the change cascades through `STG_CUSTOMERS` to marts on rebuild.
- **Audit:** the reconciliation log shows `OK — erased & suppressed` for the hero, `OK — requested, not yet actioned` for the pending pair, and `NO MATCH` for a stale ID.

Honest scope

This is a demonstration build on synthetic data. The synthetic email is non-deliverable (`[at]example.com`, RFC 2606), and no real PII exists anywhere in the warehouse. One production consideration is noted but not implemented here: if `RAW` is re-synced from source, erasure must run as a post-load step each sync — the registry makes that safe and idempotent.